

Name : Manaswi Kabadi

Class : D20C

Roll no. : 15

Blockchain Experiment - 2

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.
2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).
3. **Hashing** – SHA-256 algorithm ensures data immutability.
4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.
5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Program:

```
from flask import Flask, jsonify, request
import hashlib, json
from datetime import datetime
class Blockchain:
    def __init__(self):
        self.chain=[]
        self.pending=[]
        self.create_block(1,"0")
    def
create_block(self,proof,previous_hash):
    block={"index":len(self.chain)+1,"timestamp":str(datetime.now()),
"transactions":self.pending.copy(),"
proof":proof,
"previous_hash":previous_hash}
        self.chain.append(block)
        return block
    def
add_transaction(self,sender,receiver,amount):
    self.pending.append({"sender":sender,"receiver":receiver,"amount":amount})
    def previous_block(self):
        return self.chain[-1]
    def find_proof(self,old_proof):
        proof=1
```

```

        while True:
value=proof**2-old_proof**2

result=hashlib.sha256(str(value).encode()).hexdigest()
        if
result.startswith("000"):
            return proof
            proof+=1
        def get_hash(self,block):
data=json.dumps(block,sort_keys=True)
        )
            return
hashlib.sha256(data.encode()).hexdigest()
        def valid_chain(self):
            for i in
range(1,len(self.chain)):
previous=self.chain[i-1]
                current=self.chain[i]
                if
current["previous_hash"]!=self.get_hash(previous):
                    return False

value=current["proof"]**2-previous["proof"]**2
result=hashlib.sha256(str(value).encode()).hexdigest()
                if not
result.startswith("000"):
                    return False
                return True
app=Flask(__name__)
blockchain=BlockChain()
@app.route("/transaction",methods=["POST"])
def add_transaction():
    data=request.get_json()
    if not data or not all(k in data
for k in
["sender","receiver","amount"]):

```

```

        return
 jsonify({"error":"sender, receiver
and amount are required"}),400

blockchain.add_transaction(data["sender"],data["receiver"],data["amount"])
        return
 jsonify({"message":"Transaction
added successfully",

"pending_transactions":blockchain.pending}),201

@app.route("/mine_block",methods=["GET"])
def mine_block():
    if not blockchain.pending:
        return
 jsonify({"message":"Cannot mine. No
pending transactions."}),400

previous=blockchain.previous_block()
proof=blockchain.find_proof(previous["proof"])
block=blockchain.create_block(proof,blockchain.get_hash(previous))
        blockchain.pending.clear()
        return
 jsonify({"message":"Congratulations,
you just mined a block!",

"index":block["index"],"timestamp":block["timestamp"],

"proof":block["proof"],"previous_hash":block["previous_hash"],
"transactions":block["transactions"]}
)
@app.route("/get_chain",methods=["GET"])
def get_chain():

```

```

        return

 jsonify({"chain":blockchain.chain,"1
length":len(blockchain.chain)})
@app.route("/is_valid",methods=["GET
"])
def is_valid():
    return jsonify({"message":"All
good. The Blockchain is valid."

if
blockchain.valid_chain()
else "The
Blockchain is not valid."})
if __name__=="__main__":
app.run(host="127.0.0.1",port=5000)

```

Output :

The screenshot shows a code editor with a file named `blockchain.py`. The code defines a Flask application with two endpoints: `/get_chain` and `/is_valid`. The `/is_valid` endpoint checks the validity of the blockchain and returns a JSON response. The terminal output shows the command `py blockchain.py` being executed, and the application running on `http://127.0.0.1:5000`. The terminal also displays several HTTP requests and responses, including `POST /transaction HTTP/1.1` and `GET /get_chain HTTP/1.1`.

```

140 def get_chain():
142     return jsonify({
143         "chain": blockchain.chain,
144         "length": len(blockchain.chain)
145     })
146
147
148 # Validate chain
149 @app.route("/is_valid", methods=["GET"])
150 def is_valid():
151
152     if blockchain.valid_chain():
153         message = "All good. The Blockchain is valid."
154     else:
155         message = "The Blockchain is not valid."
156
157     return jsonify({
158         "message": message
159     })
160
161
162 if __name__ == "__main__":
163     app.run(host="127.0.0.1", port=5000)

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

PS E:\OneDrive\Desktop\BC> py blockchain.py
PS E:\OneDrive\Desktop\BC> py blockchain.py
* Serving Flask app 'blockchain'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [28/Aug/2026 19:02:53] "POST /transaction HTTP/1.1" 201 -
127.0.0.1 - - [28/Aug/2026 19:05:41] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [28/Aug/2026 19:07:31] "POST /transaction HTTP/1.1" 201 -
127.0.0.1 - - [28/Aug/2026 19:08:25] "POST /transaction HTTP/1.1" 201 -
127.0.0.1 - - [28/Aug/2026 19:08:50] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [28/Aug/2026 19:09:53] "POST /transaction HTTP/1.1" 201 -
127.0.0.1 - - [28/Aug/2026 19:10:08] "GET /get_chain HTTP/1.1" 200 -

```

HTTP <http://127.0.0.1:5000/transaction> Save Share

POST <http://127.0.0.1:5000/transaction> Send

Docs Params Authorization Headers (9) **Body** Scripts Settings Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** Schema Beautify

```
1 {
2   "sender": "Manaswi",
3   "receiver": "Ronit",
4   "amount": 20
5 }
```

Body Cookies Headers (5) Test Results 201 CREATED 12 ms 293 B 🌐 ⋮

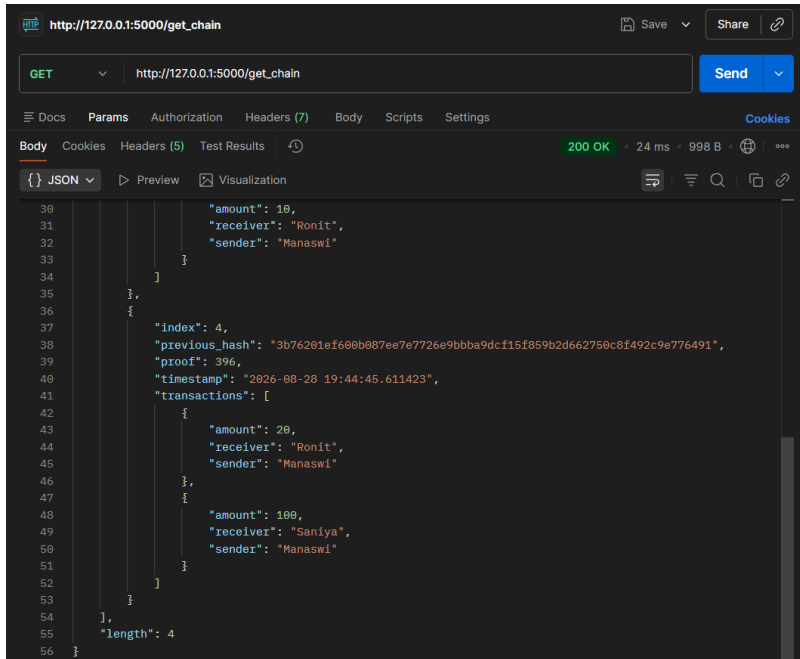
{} **JSON** Preview Visualization 🔍 📄 🔗

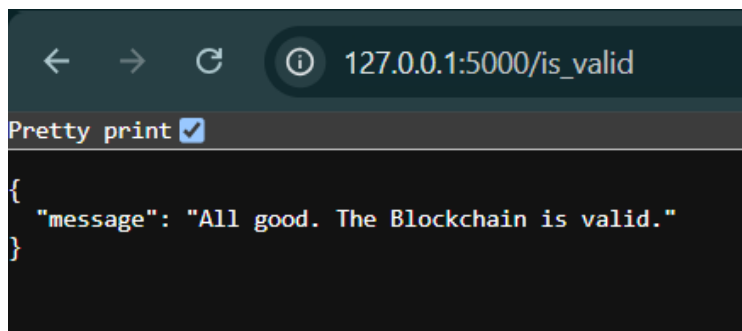
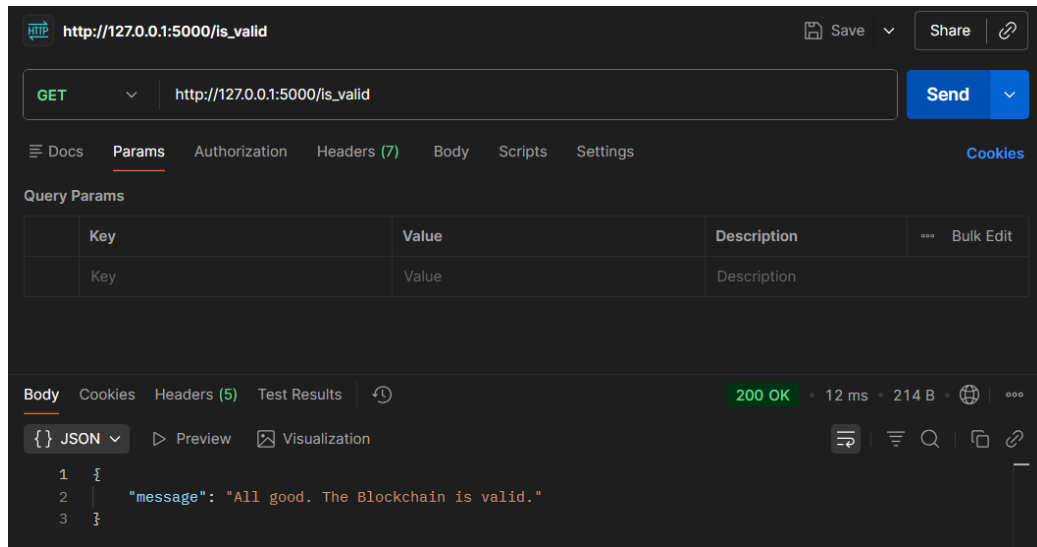
```
1 {
2   "message": "Transaction added successfully",
3   "pending_transactions": [
4     {
5       "amount": 20,
6       "receiver": "Ronit",
7       "sender": "Manaswi"
8     }
9   ]
10 }
```

← → ↻ ⓘ 127.0.0.1:5000/mine_block

Pretty print ☒

```
{
  "index": 3,
  "message": "Congratulations, you just mined a block!",
  "previous_hash": "bd064900c3fae7c23ba497cd6f0dfd387099ac31f6f2cc49f46975f9ccbe3fbc",
  "proof": 11028,
  "timestamp": "2026-08-28 19:36:34.792865",
  "transactions": [
    {
      "amount": 10,
      "receiver": "Ronit",
      "sender": "Manaswi"
    }
  ]
}
```





Conclusion :

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.